
Datenbanken

BITLC | Tom Selig
3. August 2026

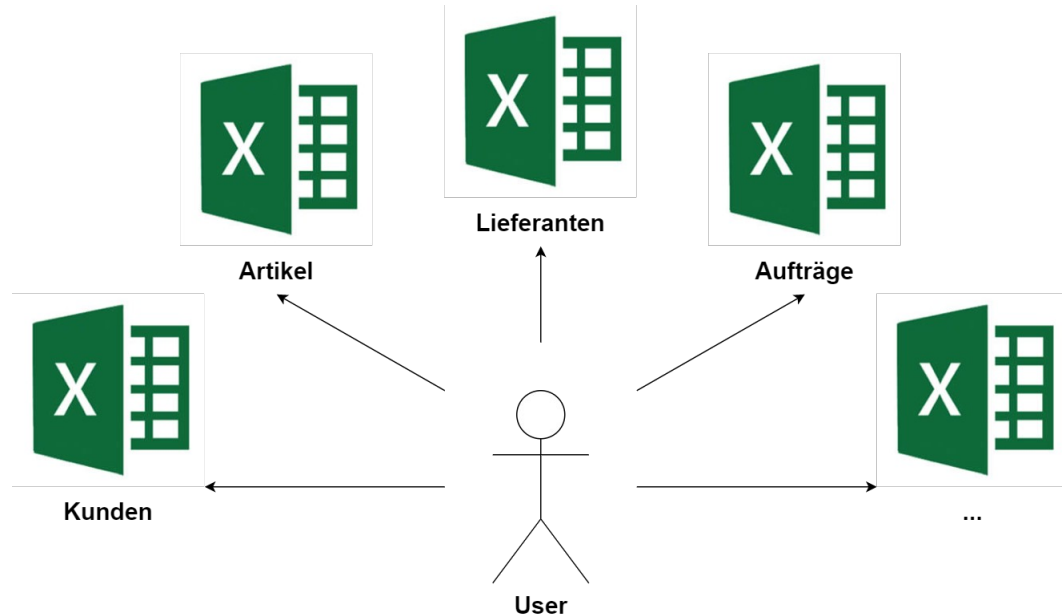
Warum Datenbanken?

- Unternehmen müssen große Mengen von Daten speichern:
Kunden, Lieferanten, Mitarbeiter, Artikel, Aufträge, ...

- Idee:

Daten werden in
Excel*-Dateien
gespeichert

* oder etwas vergleichbarem



Warum Datenbanken?

- Probleme mit Excel-Dateien:
 - Excel ist nicht netzwerkfähig
 - Daten sind nicht miteinander verknüpft
 - Zugriff durch andere Software schwierig
 - Es gibt keine Zugriffskontrolle
 - Eingeschränkte Programmierbarkeit
 - Logging und Backup nicht vorhanden
 - Keine Skalierbarkeit
- Also: Eine echte Datenbank muss her!

Grundlagen Datenbanken

- Was ist eine Datenbank?

Eine Datenbank ist eine organisierte Sammlung von Informationen oder Daten, die untereinander in einer logischen Beziehung stehen und von einem eigenen Datenbankmanagementsystem verwaltet werden.

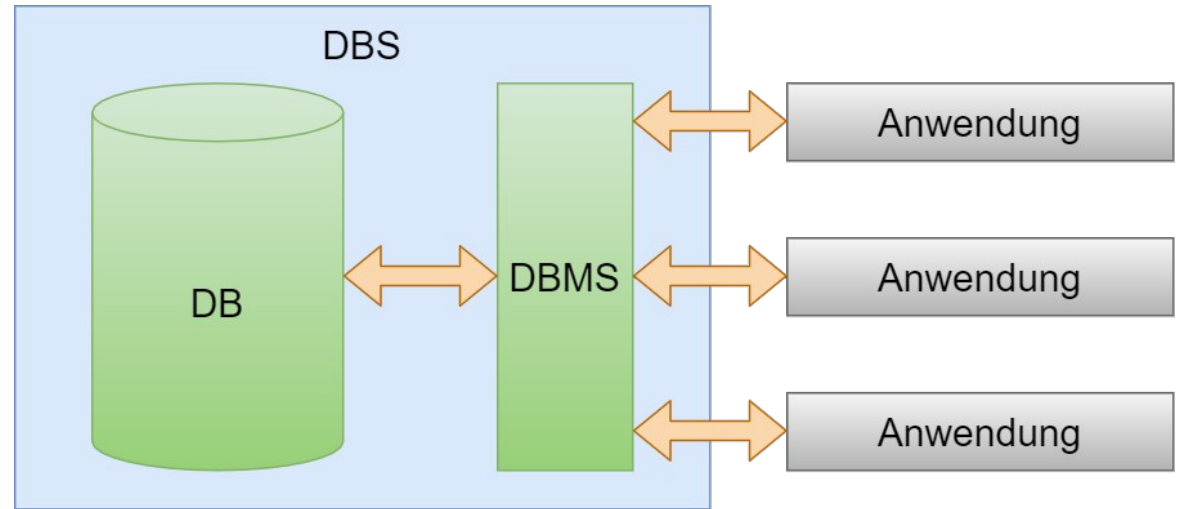
- Bestandteile eines DBS:

Datenbasis (DB):

Die eigentlichen Daten,
die gespeichert werden

Datenbankmanagement-
system (DBMS):

Software zur Verwaltung



Aufgaben eines DBMS

- Speicherung, Überschreibung und Löschung von Daten
- Verwaltung der Metadaten
- Vorkehrungen zur Datensicherheit und zum Datenschutz
- Vorkehrungen zur Datenintegrität
- Ermöglichung des Mehrbenutzerbetriebs
- Optimierung von Abfragen
- Ermöglichung von Triggern und Stored Procedures
- Bereitstellung von Kennzahlen über Technik und Betrieb

Datenbankmodelle

- Datenbankmodelle werden in zwei Kategorien eingeteilt:

Klassische Datenbankmodelle

- Etablierte, bewährte Systeme
- Für sehr viele Anwendungsfälle geeignet

NoSQL-Datenbanken

- Not only SQL
- „Neue“ Konzepte
- Erweiterung/Ergänzung der etablierten Systeme wo diese Nachteile haben

<https://db-engines.com/de/>

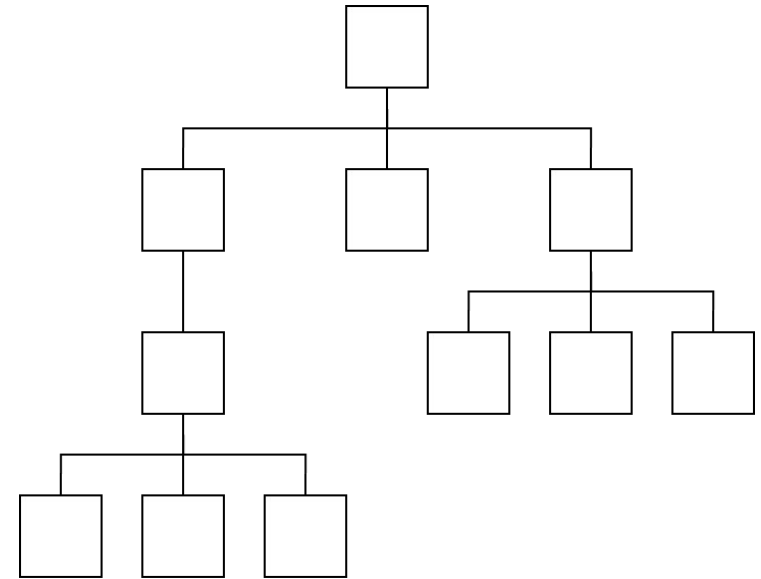
Datenbankmodelle (klassisch)

- Hierarchisches Datenbankmodell

Speicherung der Daten in hierarchischen Baumstrukturen.

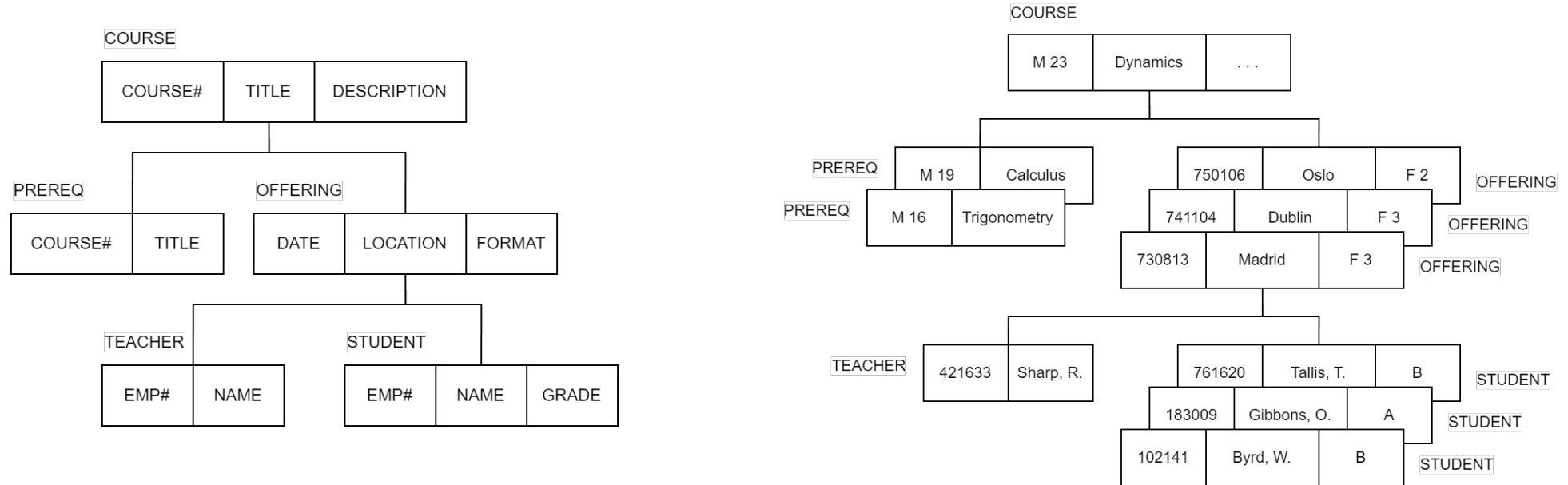
- Jeder Knoten hat maximal einen Vorgänger
- Jeder Knoten hat beliebig viele Nachfolger
- Zugriff nur über die Wurzel in Richtung des gewünschten Knotens.
- Sehr schnell, aber sehr unflexibel

z.B. IMS (IBM)



Datenbankmodelle (klassisch)

- Hierarchisches Datenbankmodell



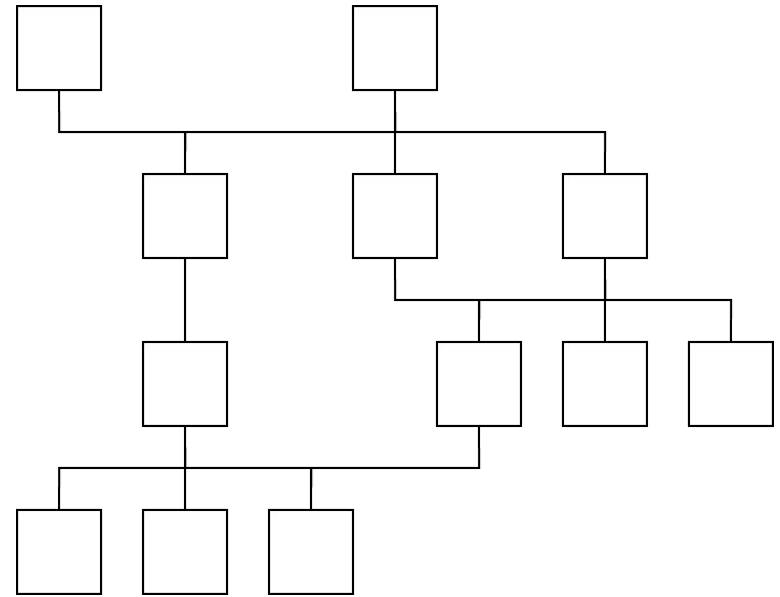
Datenbankmodelle (klassisch)

- Netzwerkdatenbankmodell

Speicherung der Daten in einer netzwerkartigen Struktur.

- Weiterentwicklung des hierarchischen Datenbankmodells
- Verbindungen nicht mehr nur hierarchisch, sondern auch Querverbindungen möglich
- Jeder Knoten kann jetzt auch mehrere übergeordnete Knoten haben

z.B. IDMS (Computer Associates)
UDS (Fujitsu/Siemens)



Datenbankmodelle (klassisch)

- Relationales Datenbankmodell

Speicherung der Daten in Tabellen (Relationen) mit festen Spalten und variablen Zeilen

- Entwickelt ca. 1970
- Relativ einfache Handhabung, daher sehr weite Verbreitung (seit Jahren führend in den Ranglisten der DB-Systeme)
- Manipulation und Abfrage der Daten mittels SQL
- Aufteilung der Daten auf Tabellen durch Normalisierung (Vermeidung von Redundanzen)
- Dadurch oft komplexe Abfragen über mehrere Tabellen hinweg

z.B. SQL Server (Microsoft), Oracle Database, MySQL/MariaDB

Datenbankmodelle (klassisch)

- Objektorientiertes Datenbankmodell

Speicherung der Daten in Form von Objekten

- Macht ORM-Tools im Bereich der objektorientierten Programmierung überflüssig
- Objekte können ohne Umformung gespeichert werden
- Beziehungen zwischen den Objekten (z.B. Vererbungen) bleiben erhalten
- Auch komplexe Objekte werden in atomaren Operationen verarbeitet
- Sehr komplexer Aufbau und Wartung

z.B. O2 (O2 Technology) und ObjectStore (Ignite Technologies)

Datenbankmodelle (NoSQL)

- NoSQL-Datenbanken definieren sich in erster Linie durch das, was sie anders machen wollen als relationale Datenbanken:

Ein einfaches, nicht relationales Datenmodell
(schemalose Datenspeicherung)

Einfacher Zugriff auf die Daten ohne SQL
(auch wenn einige Vertreter das z. T. ermöglichen)

Hohe horizontale Skalierbarkeit
(Verteilung der Daten über mehrere Rechner)

- Die Entwicklung erfolgte parallel zum Aufstieg der großen Internetunternehmen wie Amazon, Facebook, Google, etc.

Relationale Datenbanken kamen an Ihre Grenzen (und noch weiter)

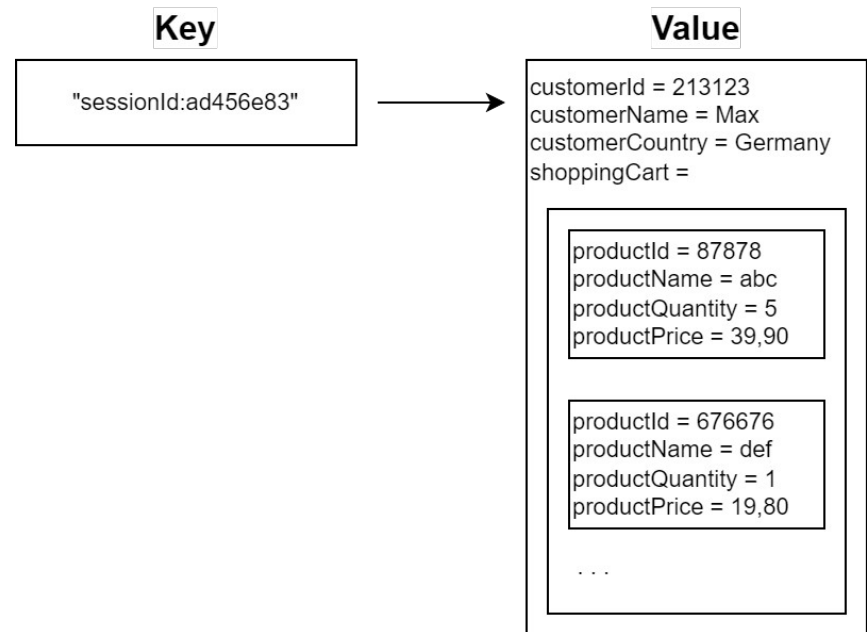
Datenbankmodelle (NoSQL)

- Key-Value Stores

Speichern Daten in Form von Schlüssel-Wert-Paaren.

- Zugriff ist nur über den Schlüssel (Key) möglich.
- Values können beinahe beliebigen Aufbau haben
- Schlüssel sind häufig Strings aus mehreren Teilen
- Sehr einfache, dafür aber performante Systeme.

*z.B. Redis, Amazon DynamoDB,
Microsoft Azure Cosmos DB*



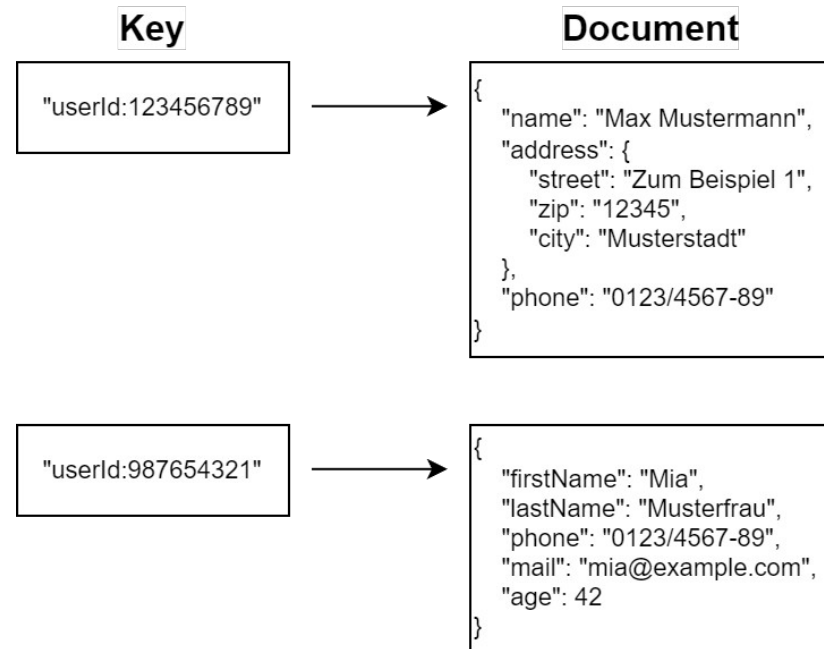
Datenbankmodelle (NoSQL)

- Document Stores

Speichern Daten in schemafreien Dokumenten.

- Sehr flexibles System, da keine starre Struktur vorgegeben.
- Dokumente können beliebigen, unterschiedlichen Aufbau haben
- Daten werden intern häufig im JSON-Format gespeichert.
- Suche auch innerhalb der Dokumente möglich

z.B. MongoDB, Couchbase, Firebase (Google)



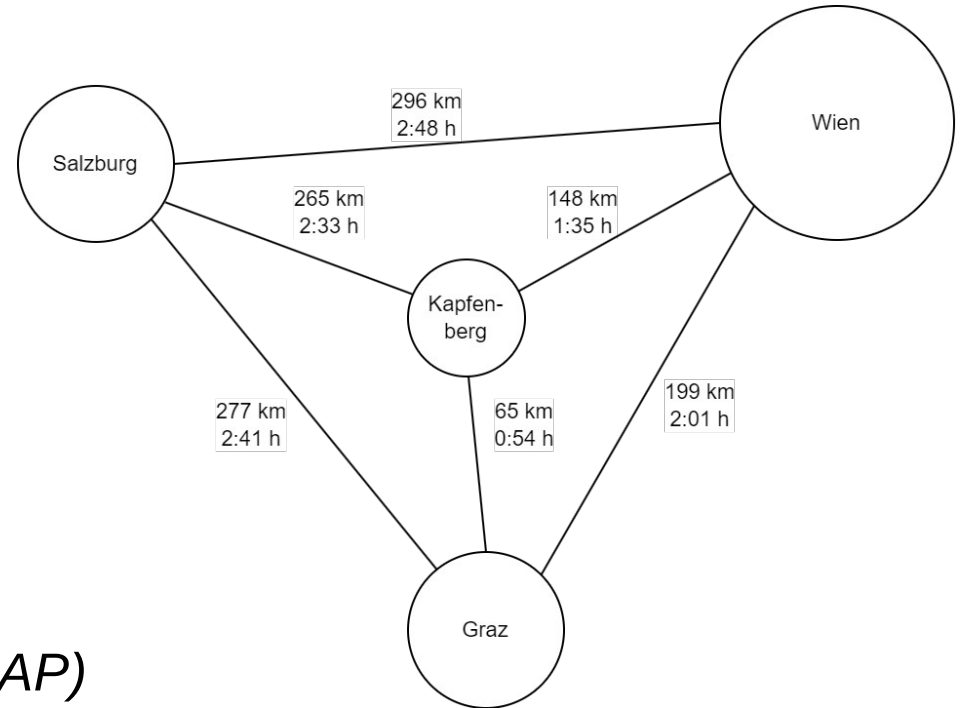
Datenbankmodelle (NoSQL)

- Graphdatenbanken

Stellen Datensätze in Form von Knoten und Kanten dar.

- Im Vordergrund stehen die Beziehungen der Knoten zueinander.
- Kanten können gerichtet sein und Gewichtungen haben.
- Direkter Zugriff über Eigenschaften der Knoten meist nicht möglich.

z.B. Neo4j, ArangoDB, OrientDB (SAP)



Datenbankmodelle (NoSQL)

- Wide Column Stores (Spaltenorientierte Datenbanken)

Speichern Daten in Datensätzen mit extrem vielen dynamischen Spalten.

- Ähnlich anzusehen wie zweidimensionale Key-Value Stores.
- Zur Verwaltung riesiger Datenmengen über tausende Server hinweg.

*z.B. Big Table (Google),
Cassandra (Apache),
HBase (Apache)*

Row Key				
Row "user:123456789"	name	street	zip	city
	Mustermann	Zum Beispiel 1	12345	Musterstadt
Row "user:987654321"	name	phone	mail	
	Musterfrau	0123/4567-89	m@xmpl.com	
Row "user:543219876"	name	age		
	Tester	23		

Datenbankmodelle (NoSQL)

- Content Stores

Speichern digitale Inhalte wie z.B. Texte, Bilder, Videos mit ihren zugehörigen Metadaten

- Unterstützung von Volltextsuche, Versionierung und Zugriffssteuerung
- Abbildung von hierarchischen Strukturen
- Abfragen über SQL und/oder XPath möglich
- Bilden häufig die Basis für ein Content- oder Document-Management-System

z.B. Jackrabbit (Apache), ModeShape

Datenbankmodelle (NoSQL)

- Suchmaschinen

Spezialisiert auf die Suche von Dateninhalten

- Unterstützung von Volltextsuche und Stemming (Stammformreduktion)
- Verarbeitung auch komplexer Suchbegriffe
- Möglichkeit der Ergebnisreihung
- Gruppierung von Ergebnissen
- Unterstützung verteilter Suche für hohe Skalierbarkeit

z.B. Elasticsearch (Elastic), OpenSearch (Amazon)

Datenbankmodelle (NoSQL)

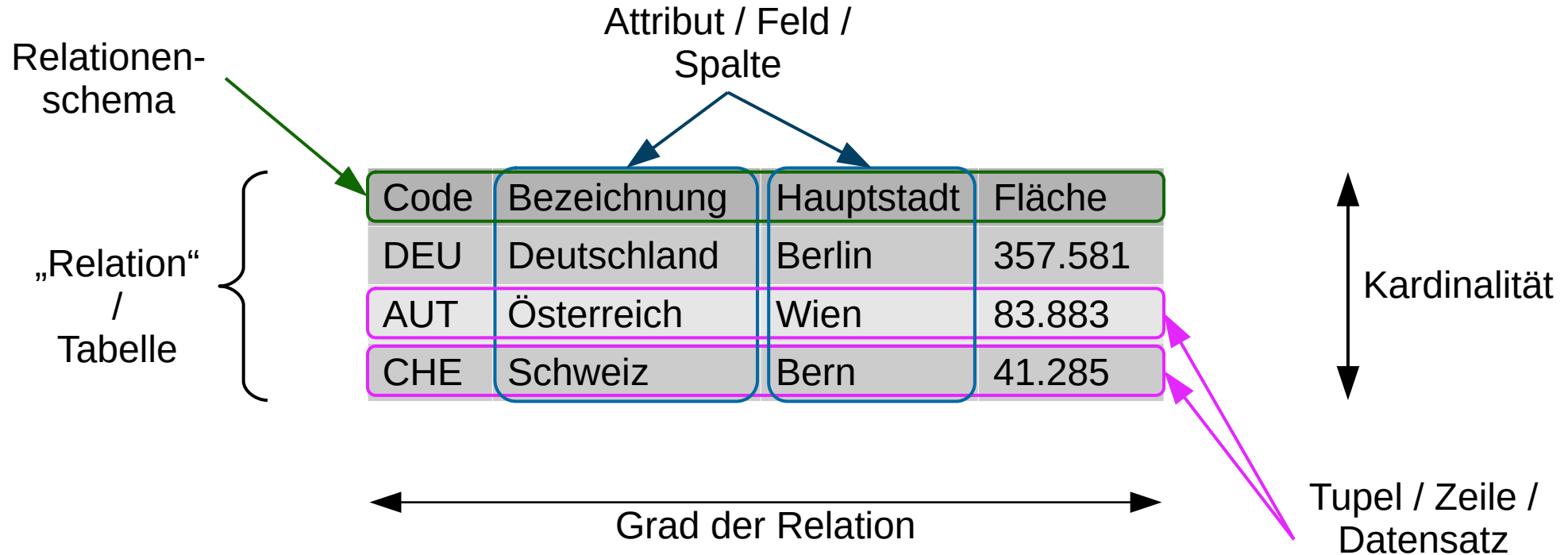
- Anwendungsfälle für NoSQL-Systeme:
 - Suchindex für das Internet (Google, Bing, ...)
 - Soziale Netzwerke (Facebook, Twitter, TikTok, ...)
 - E-Mail- und Dokumenten-Server (Gmail, Microsoft 365, ...)
 - Cloudspeicher (Dropbox, OneDrive, Google Drive, ...)
 - Versionsverwaltungssysteme (GitHub, GitLab, ...)
 - Multi-User-Games (World of Warcraft, ...)
 - Künstliche Intelligenz (selbstfahrende Autos, ...)
 - ...

Relationale Datenbanken

Grundlagen

- Konzept wurde entwickelt von Dr. Edgar Frank Codd (1970).
- Daten werden in Tabellen (Relationen) gespeichert, die zueinander in Beziehung stehen.
- Tabellen haben eine feste Struktur, d.h. eine feste Anzahl an Spalten (Attributen), aber eine variable Anzahl an Zeilen (Tupeln).
- Einzelne Zeilen der Tabelle repräsentieren die Datensätze und bestehen aus Werten für die einzelnen Attribute.
- Die Beziehungen zwischen den einzelnen Tabellen werden über (Fremd-)Schlüssel hergestellt.

Grundlagen



Mengen

- Eine Relation besteht aus einem Kopf und einem Rumpf. Der Kopf ist eine Menge von Attributen, der Rumpf eine Menge von Tupeln.
- Was ist eigentlich eine Menge?

Unter einer Menge verstehen wir jede Zusammenfassung M von bestimmten, wohlunterschiedenen Objekten (m) unserer Anschauung oder unseres Denkens (welche die Elemente von M genannt werden) zu einem Ganzen. Georg Cantor

- Daraus folgt:

Eine Menge wird stets als Ganzes betrachtet.

Es gibt keine Dubletten.

Es gibt keine Reihenfolge.

Mengen

- Beispiele für Mengen:

$$A = \{ 1, 2, 3, 4 \}$$

$$B = \{ 25, -16, 9, \pi \}$$

$$C = \{ \text{Karo, Herz, Pik, Kreuz} \}$$

$$D = \{ a, b, e, m \}$$

$$\emptyset = \{ \}$$

- Nach der Mengendefinition gilt dabei:

$$\{ 1, 2, 5 \} = \{ 5, 1, 2 \}$$

Die Elemente einer Menge haben keine besondere Reihenfolge.

$$\{ 1, 2, 5 \} = \{ 1, 2, 2, 5, 2, 1 \}$$

Es gibt keine Dubletten in einer Menge, ein Element ist entweder in einer Menge enthalten oder nicht.

Mengen

- Die Anzahl an Elementen einer Menge wird als Mächtigkeit der Menge bezeichnet:

$$A = \{ 1, 2, 3, 4 \} \quad \Rightarrow \quad |A| = 4$$

$$\emptyset = \{ \} \quad \Rightarrow \quad |\emptyset| = 0$$

- Auf Mengen können Operationen angewendet werden:

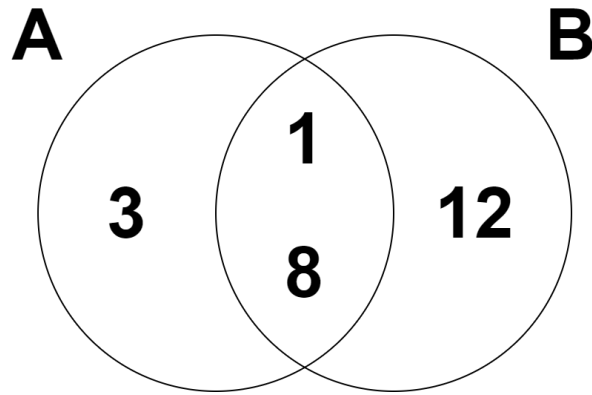
Vereinigung

Schnitt

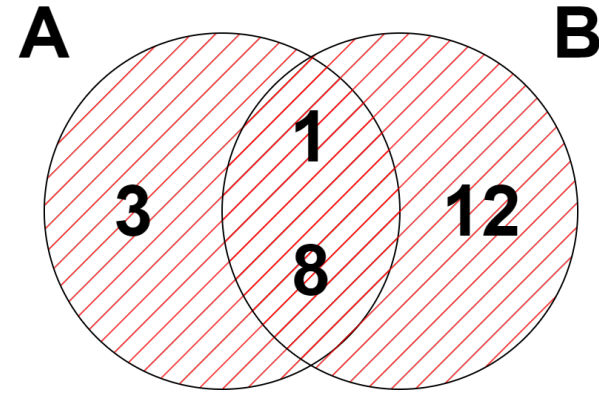
Differenz

Vereinigungsmenge

$$A = \{ 1, 3, 8 \}, B = \{ 1, 8, 12 \}$$



$$A \cup B = \{ 1, 3, 8, 12 \}$$



Vereinigung zweier Mengen:

Menge aller Elemente, die in mindestens einer der beiden Mengen enthalten sind.

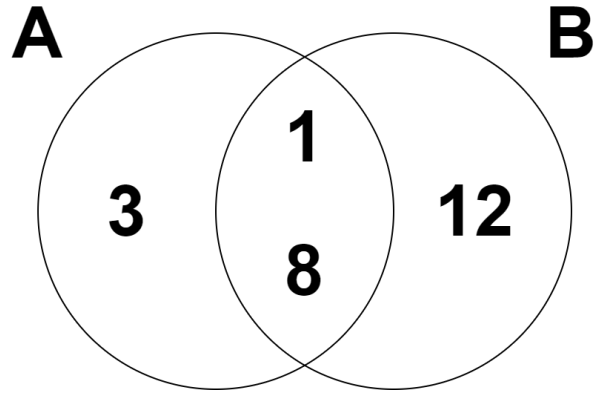
$$x \in A \cup B$$

$\Leftrightarrow$

$$x \in A \vee x \in B$$

Schnittmenge

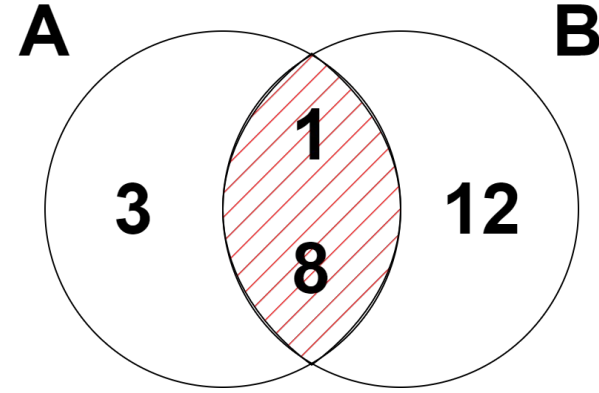
$$A = \{ 1, 3, 8 \}, B = \{ 1, 8, 12 \}$$



Schnitt zweier Mengen:

Menge aller Elemente, die in beiden Mengen enthalten sind.

$$A \cap B = \{ 1, 8 \}$$



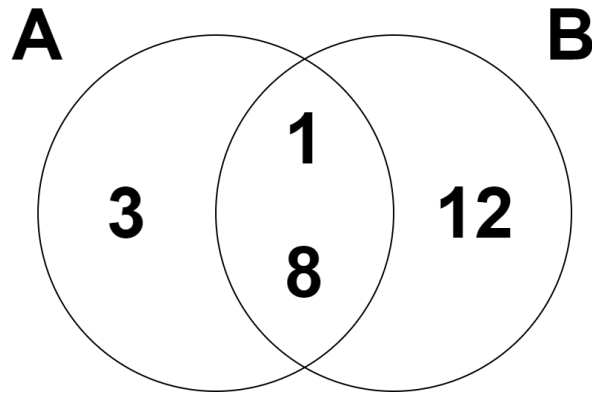
$$x \in A \cap B$$

$\Leftrightarrow$

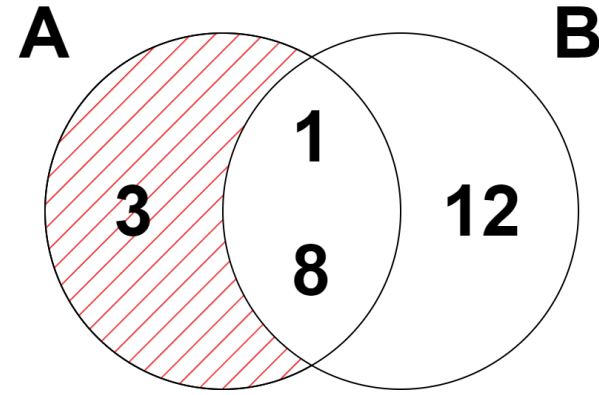
$$x \in A \wedge x \in B$$

Differenzmenge

$$A = \{ 1, 3, 8 \}, B = \{ 1, 8, 12 \}$$



$$A \setminus B = \{ 3 \}$$



Differenz zweier Mengen:

Menge aller Elemente, die in der einen Menge, aber nicht in der anderen enthalten sind.

$$x \in A \setminus B$$

$\Leftrightarrow$

$$x \in A \wedge x \notin B$$

Gleichheit / Teilmengen

- Gleichheit von Mengen:

Für jedes Element gilt: Wenn es in A ist, dann ist es auch in B und umgekehrt.

$$A = B \Leftrightarrow (x \in A \Leftrightarrow x \in B)$$

- Teilmenge

Für jedes Element gilt: Wenn es in A ist, dann ist es auch in B .

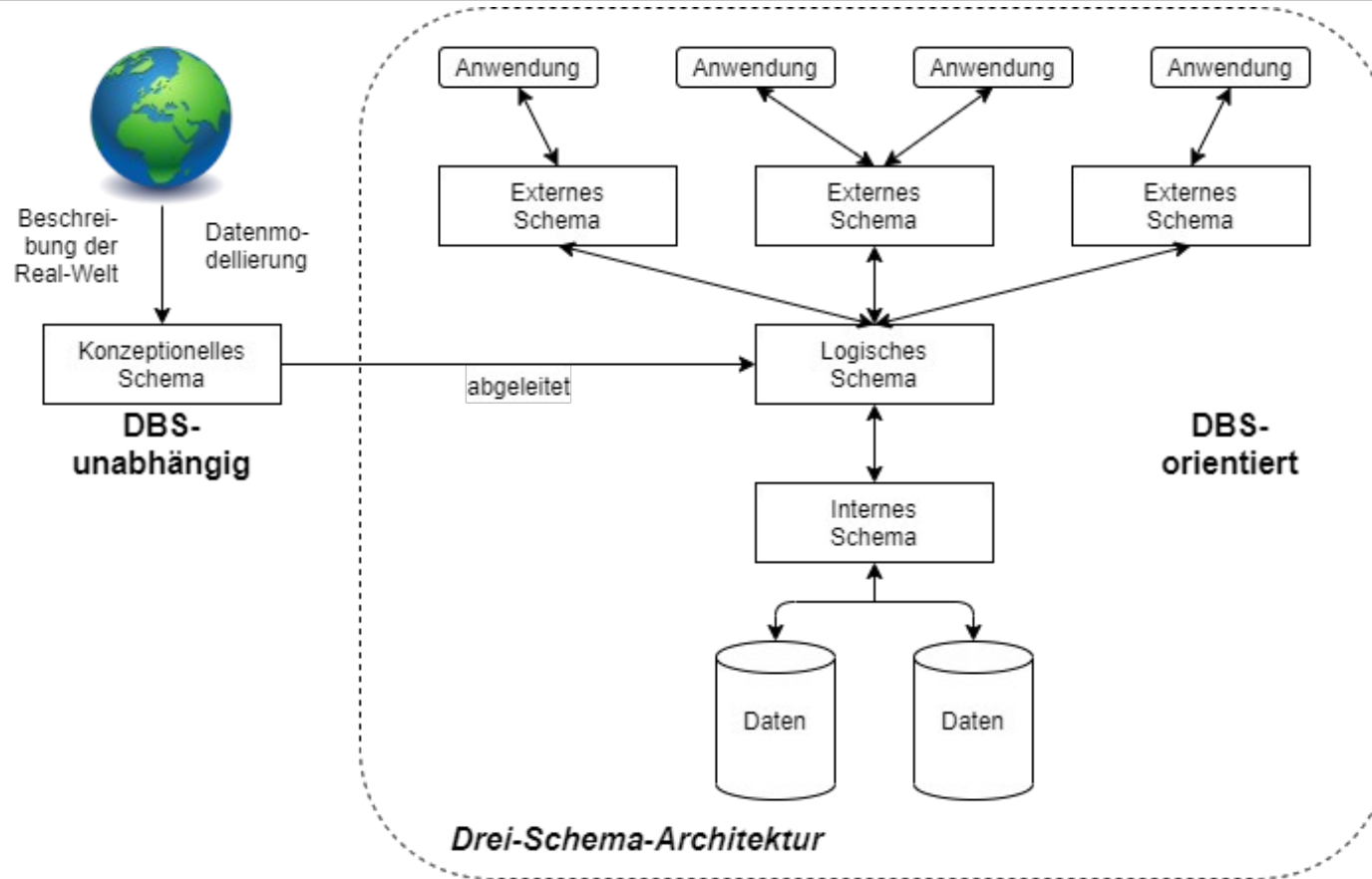
$$A \subseteq B \Leftrightarrow (x \in A \Rightarrow x \in B)$$

- Echte Teilmenge

A ist Teilmenge von B , aber A ist nicht gleich B .

$$A \subset B \Leftrightarrow (A \subseteq B \wedge A \neq B)$$

Schemas für relationale Datenbanken



Schemas für relationale Datenbanken

- Konzeptionelles Schema

Systemunabhängige Datenbeschreibung, orientiert sich ausschließlich an der realen Welt.

Muss nicht die Besonderheiten eines speziellen DBMS berücksichtigen.

Z.B.: m:n-Beziehungen können dargestellt werden.

- Logisches Schema

Beschreibt die Daten in der Sprache eines konkreten DBMS.

Abgeleitet aus dem konzeptionellen Schema.

Z.B.: m:n-Beziehungen müssen aufgelöst werden.

Schemas für relationale Datenbanken

- Internes/Physikalisches Schema

Beschreibt die Daten aus technischer Sicht des Systems.

Abgeleitet aus dem logischen Schema.

Z.B.: Speichermethoden, Hilfskonstrukte zur Steigerung der Effizienz.

- Externes Schema

Beschreibt die Daten aus der Sicht des Anwenders.

Einschränkung der Sichtbarkeit, Vergabe von Zugriffsrechten.

Es kann mehrere externe Schemas zu einer Datenbank geben.

Z.B.: Anwendungsprogramm, das auf die Datenbank zugreift.